UNIVERSIDAD DE SAN CARLOS DE GUATEMALA, CENTRO UNIVERSITARIO DE OCCIDENTE CUNOC

Teoría de Sistemas 1

Segundo Semestre 2026

Documento Final del Proyecto

Claude Lovers Gym — Sistema de administración de membresías de gimnasio

Integrantes

Carlos Raúl Alberto López Peláez 202031871
Diego Alessandro González Bravo 202230939
Enmer Oswaldo Sandoval Mazariegos
Erikson Denilson Orozco Monterroso 202131770
Brandon David Cotom Tezó 202231460

29 de octubre de 2026

1. Introducción

Claude Lovers Gym nació como respuesta a un problema concreto: los gimnasios pequeños y medianos suelen administrar sus membresías con hojas de cálculo, cuadernos o sistemas de punto de venta genéricos que no están pensados para el ciclo de vida particular de una membresía su vencimiento, su renovación, su posible congelamiento temporal, o el cobro en abonos cuando un socio no puede pagar de contado. El proyecto se propuso construir un sistema propio, desplegado en la nube, que cubriera ese ciclo completo: desde que un socio se inscribe por primera vez hasta que paga, asiste, eventualmente congela su membresía por un viaje o una lesión, y finalmente la renueva o la cancela.

El desarrollo se organizó en seis sprints a lo largo del semestre, cada uno con un foco distinto: fundamentos y autenticación, el núcleo de socios y membresías, la automatización del despliegue (CI/CD), el módulo de pagos, los ajustes del Product Owner sobre precios y recargos, y finalmente el cierre funcional con congelamiento de membresías, pagos en cuotas y reportes. Este documento reúne, para la entrega final, el resultado de ese trabajo: los requerimientos del sistema, su arquitectura y modelo de datos, el historial de lo entregado en cada sprint con los problemas reales que se presentaron y cómo se resolvieron, una retrospectiva honesta del equipo, y una comparación entre lo que se estimó y lo que realmente tomó construir cada pieza.

2. Documento de requerimientos

2.1 Propósito

El sistema tiene como propósito ofrecer al gimnasio una herramienta única para gestionar la membresía de sus socios de principio a fin: alta del socio, asignación y ciclo de vida de su plan incluyendo congelamientos, cobro de sus cuotas de contado o en abonos, control de acceso físico mediante check-in y check-out, y visibilidad del negocio a través de reportes con datos reales, reemplazando así el control manual que antes se llevaba en hojas de cálculo independientes.

2.2 Alcance

El alcance de esta entrega incluye: autenticación y manejo de roles (Administrador, Recepcionista, Entrenador y Socio), gestión de socios y sucursales, catálogo y asignación de planes de membresía, congelamiento temporal de membresías, registro de pagos completos y en cuotas con generación de comprobante en PDF, control de asistencia, notificaciones y recordatorios de vencimiento, promociones y recargos configurables, auditoría de cambios, configuración de reglas del negocio, y reportes operativos y financieros.

Quedan explícitamente fuera del alcance de esta entrega la gestión de clases grupales, las rutinas de entrenamiento personalizadas y las mediciones físicas periódicas de los socios. El esquema de base de datos reserva tablas para esas funcionalidades pensando en una fase posterior del proyecto, pero actualmente no tienen controlador, servicio ni interfaz implementados; mencionarlas aquí evita que se interprete su ausencia como un descuido en lugar de una decisión de alcance.

2.3 Requerimientos funcionales

A continuación se listan los requerimientos funcionales agrupados por módulo. Cada uno conserva el identificador (RF-NN) con el que se referencia desde el historial de sprints y la tabla de estimación de tiempos, de modo que pueda rastrearse en qué sprint se implementó cada uno y cuánto tiempo tomó.

Autenticación y usuarios

RF-01 — Inicio de sesión. El usuario introduce su correo y contraseña; el sistema valida las credenciales contra la base de datos y, si son correctas, emite un token JWT firmado. El token expira a las 24 horas (variable JWT_EXPIRATION); cualquier acción protegida con un token vencido devuelve 401 y obliga a iniciar sesión de nuevo.

RF-02 — Gestión de usuarios y roles. Un Administrador puede crear, editar, desactivar y reactivar cuentas de Recepcionista, Entrenador y otros Administradores, asignándoles el rol correspondiente desde el catálogo fijo (ADMIN, RECEPCIONISTA, ENTRENADOR, CLIENTE).

Socios y sucursales

RF-03 — Alta y edición de socios. Recepción registra los datos personales de un socio nuevo (nombres, correo, teléfono, DPI, fecha de nacimiento) y el sistema crea automáticamente su cuenta de usuario con rol CLIENTE; los datos pueden editarse después y el socio puede darse de baja (soft delete) sin perder su historial.

RF-20 — Gestión de sucursales. El Administrador mantiene el catálogo de sucursales (alta, desactivación y reactivación) y puede asignar una membresía a una o varias sucursales simultáneamente (soporte multisucursal).

Planes y membresías

RF-04 — Catálogo de planes. El sistema mantiene un catálogo de planes de membresía con nombre, duración en días y precio; los planes pueden consultarse desde cualquier pantalla donde se asigna o cobra una membresía.

RF-05 — Asignación de membresía. Al asignar un plan a un socio se crea una membresía con estado ACTIVA; la fecha de vencimiento se calcula automáticamente sumando la duración del plan a la fecha de inicio, sin intervención manual.

RF-06 — Edición y cancelación. Una membresía puede editarse (cambiar de plan, ajustar fechas) o cancelarse indicando un motivo; la cancelación admite marcar si corresponde reembolso y por qué monto.

RF-07 — Congelamiento temporal. Recepción o el Administrador pueden congelar una membresía activa indicando la fecha hasta la que estará congelada y un motivo. El sistema valida que no exista ya un congelamiento traslapado, que el número de días esté dentro del mínimo y el máximo configurados, y que no se exceda el máximo de congelamientos permitidos por año. Al registrarse, los días congelados se suman a la fecha de vencimiento; si se descongela antes de la fecha programada, se descuentan del vencimiento los días que no llegaron a usarse.

Pagos y facturación

RF-08 — Registro de pago. Recepción registra el pago de una membresía indicando el monto, el método de pago y, opcionalmente, una promoción o un recargo por mora; el sistema calcula el total, genera la factura y el comprobante, y recalcula la fecha de vencimiento de la membresía.

RF-09 — Pago en cuotas (abonos). Si el pago registrado no cubre el total de la factura, esta queda con estado PENDIENTE y un saldo; Recepción puede registrar abonos adicionales contra esa factura hasta saldarla por completo, momento en el que pasa automáticamente a estado PAGADA.

RF-10 — Estado de cuenta. Tanto el personal como el propio socio pueden consultar el estado de cuenta de un socio: total facturado, total pagado, saldo pendiente, parte del saldo ya vencida y la fecha límite más próxima para liquidar.

RF-11 — Comprobante en PDF. Cada pago, completo o parcial, genera un comprobante descargable en PDF con el detalle del socio, el plan, el monto, el método de pago y quién lo atendió.

RF-12 — Historial de pagos. El historial de pagos de un socio puede filtrarse por rango de fechas y por método de pago, tanto desde la vista administrativa como desde el portal del propio socio.

RF-13 — Cotización previa. Antes de confirmar un cobro, el sistema permite cotizar el monto final aplicando el mejor descuento vigente o un recargo por pago tardío, para que Recepción confirme el monto con el socio antes de registrar el pago.

RF-22 — Promociones. El Administrador configura promociones (porcentaje o monto fijo de descuento) con vigencia por fechas; al cotizar o registrar un pago, el sistema aplica automáticamente la promoción vigente más favorable si no se indica una explícitamente.

Asistencia y control de acceso

RF-14 — Registro de entrada. Al marcar el ingreso de un socio, el sistema valida que tenga una membresía ACTIVA y vigente en ese momento; si no la tiene, rechaza el ingreso con un mensaje explicando la razón.

RF-15 — Registro de salida. Recepción puede cerrar el registro de asistencia del día marcando la hora de salida del socio.

RF-16 — Historial e indicadores. El sistema conserva el historial de asistencias por socio y expone indicadores agregados (visitas por día, horas pico) usados en los reportes.

Vencimientos y notificaciones

RF-17 — Detección de vencimientos próximos. El sistema identifica las membresías cuya fecha de vencimiento cae dentro de un número de días configurable (por ejemplo, 7, 15 o 30) y las expone en un panel dentro de la aplicación para que el personal pueda contactar a esos socios antes de que venzan.

RF-18 — Recordatorio por correo. Desde el panel de vencimientos, el personal puede enviar al socio un correo de recordatorio con la fecha de vencimiento y los días restantes.

RF-19 — Notificaciones internas. El personal recibe notificaciones dentro de la aplicación sobre eventos relevantes (por ejemplo, membresías vencidas o facturas con saldo vencido).

Auditoría, configuración y reportes

RF-21 — Bitácora de auditoría. Toda modificación sobre una membresía o una sucursal (quién la hizo, cuándo y qué cambió) queda registrada en una bitácora consultable.

RF-23 — Configuración de reglas. El Administrador puede ajustar, desde la aplicación y sin tocar código, los parámetros de negocio: días mínimo y máximo de congelamiento, máximo de congelamientos por año y el monto o porcentaje del recargo por pago tardío.

RF-24 — Reportes operativos. El sistema expone reportes con datos reales de la base de datos: ingresos por mes, miembros activos frente a morosos, renovaciones, distribución de socios por plan y por sucursal, y estadísticas de asistencia.

Portales de autoservicio

RF-25 — Portal del socio. El socio, al iniciar sesión, ve su propia membresía (plan, vigencia, días restantes), su estado de cuenta si tiene saldo pendiente, su historial de pagos con acceso a los comprobantes, y un aviso visible si su membresía está por vencer o ya venció.

RF-26 — Espacio del entrenador. El entrenador, al iniciar sesión, consulta la lista de socios que tiene asignados.

2.4 Requerimientos no funcionales

Además de las funcionalidades visibles para el usuario, el sistema debe cumplir las siguientes condiciones medibles, que se verificaron durante el desarrollo y la operación del sistema desplegado:

RNF-01. La sesión del usuario expira a las 24 horas de haberse emitido el token (parámetro JWT_EXPIRATION); vencida, cualquier acción protegida exige iniciar sesión de nuevo.

RNF-02. Las contraseñas se almacenan con hash BCrypt; en ningún punto del sistema, ni en la base de datos ni en los registros de la aplicación, se guarda una contraseña en texto plano.

RNF-03. Los permisos por rol se validan en el backend mediante anotaciones de seguridad (@PreAuthorize) en el cien por ciento de los endpoints sensibles, de modo que ocultar un botón en la interfaz nunca es el único mecanismo de control de acceso.

RNF-04. El pipeline de integración continua ejecuta la batería de pruebas automatizadas de cada repositorio en menos de cinco minutos, para no frenar el flujo de trabajo del equipo.

RNF-05. Un respaldo completo de la base de datos puede generarse y restaurarse en menos de un minuto con el volumen de datos de demostración actual.

RNF-06. La aplicación es utilizable desde un navegador de escritorio estándar (Chrome, Edge o Firefox actualizados) sin requerir instalación adicional de parte del usuario.

2.5 Restricciones

- El servidor de producción es una instancia AWS EC2 de tipo t3.micro (1 vCPU, 1 GB de RAM); sus recursos son insuficientes para compilar el backend y el frontend directamente ahí, lo que obligó a que el pipeline de despliegue continuo compile las imágenes de Docker en los servidores de GitHub Actions y el servidor solo las descargue ya construidas.
- El dominio público del sistema depende de un proxy Cloudflare administrado por uno de los integrantes del equipo; su configuración de cifrado SSL/TLS queda fuera del control directo de los repositorios de código.
- El backend está fijado en Java 21 sobre Spring Boot 4, y el frontend en Angular 21; ambas versiones se definieron desde el primer sprint y no se migraron durante el proyecto.

2.6 Supuestos

- Cada socio corresponde a un único usuario del sistema: la relación entre la entidad socio y la entidad usuario es de uno a uno.
- Los catálogos de métodos de pago y de estados (de membresía, de factura) son fijos y se siembran mediante migraciones de base de datos; no se gestionan desde la interfaz de usuario.
- El sistema opera en un solo idioma (español, variante de Guatemala) y una sola moneda (quetzales), sin soporte multi-idioma ni multi-moneda.

3. Arquitectura y diseño del sistema

3.1 Visión general

El sistema sigue una arquitectura de tres capas desplegada en contenedores independientes: un frontend en Angular 21 que consume una API REST expuesta por un backend en Spring Boot 4, que a su vez persiste en una base de datos MariaDB 11. Cada capa corre en su propio contenedor Docker frontend, backend y base de datos orquestados por un único archivo docker-compose.yml, de modo que todo el sistema puede levantarse desde cero con un solo comando tanto en una máquina de desarrollo como en el servidor de producción.

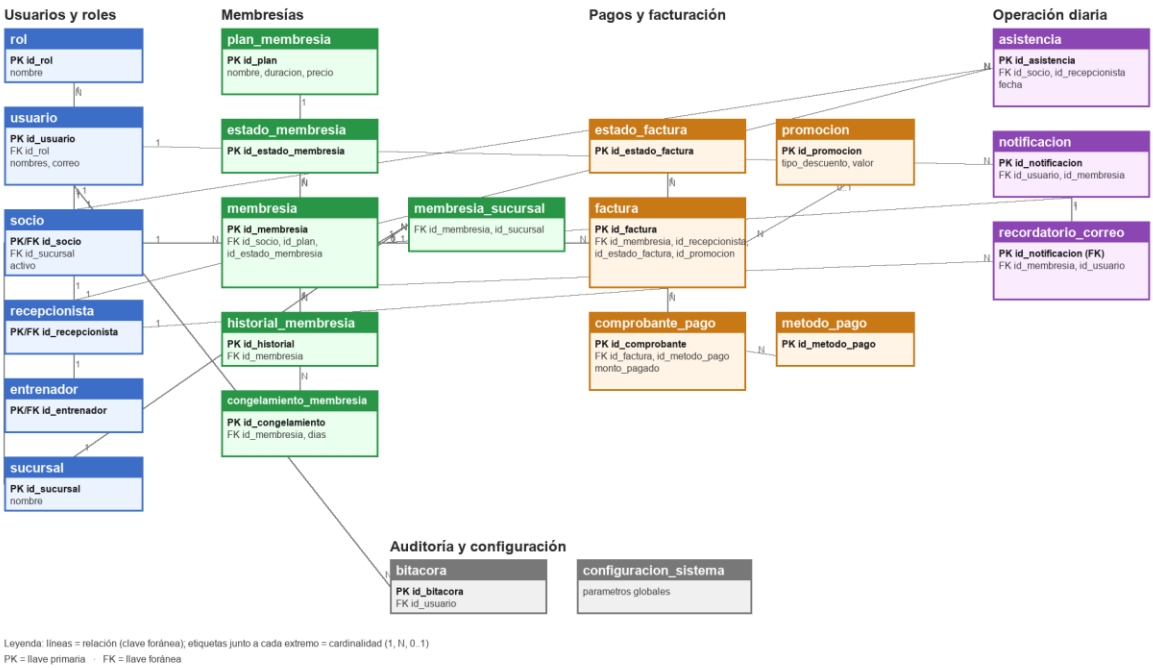
En producción, el contenedor del frontend sirve el build estático de Angular a través de Nginx, el mismo Nginx que actúa como proxy reverso hacia el backend para todas las peticiones que empiezan con /api/, de modo que el navegador nunca necesita conocer la dirección real del backend ni cruzar dominios. Por delante de ese Nginx interno hay un segundo Nginx corriendo directamente en el sistema operativo de la instancia EC2, que a su vez recibe el tráfico cifrado que llega desde Cloudflare. Esta doble capa de proxy permite que el dominio público (detrás de Cloudflare) y los puertos internos de cada contenedor se mantengan desacoplados.

El backend está organizado en capas clásicas: controladores REST que exponen los endpoints, servicios que contienen las reglas de negocio, repositorios de Spring Data JPA para el acceso a datos, y entidades JPA que modelan las tablas. Entre los controladores y las entidades se interponen DTOs y mappers dedicados, de modo que la forma en que la API expone los datos nunca queda acoplada directamente a la estructura interna de la base de datos. El frontend, por su parte, está organizado por módulo de negocio socios, membresías, pagos, asistencia, reportes, entre otros, cada uno con su propio servicio HTTP, sus propios modelos TypeScript y sus propios componentes, lo que permitió que distintos integrantes del equipo trabajaran en paralelo sobre módulos distintos durante casi todo el semestre sin generar conflictos de código.

El pipeline de integración y despliegue continuo está construido sobre GitHub Actions. Un push a la rama principal dispara primero el flujo de integración continua, que compila el proyecto y ejecuta su batería de pruebas automatizadas; si ese flujo termina exitosamente, se dispara automáticamente el flujo de despliegue continuo, que compila la imagen de Docker correspondiente directamente en los servidores de GitHub —evitando así la limitación de memoria de la instancia de producción—, la publica en GitHub Container Registry, y finalmente se conecta por SSH a la instancia EC2 para descargar esa imagen y reiniciar únicamente el contenedor que cambió.

3.2 Modelo de datos

El diagrama siguiente muestra las entidades que respaldan las funcionalidades implementadas en esta entrega, agrupadas en cinco dominios: Usuarios y roles, Membresías, Pagos y facturación, Operación diaria (asistencia y notificaciones), y Auditoría y configuración. El esquema real de la base de datos contempla además tablas para clases grupales, rutinas de entrenamiento y mediciones físicas, reservadas para una fase futura del proyecto; como esas tablas no tienen todavía ningún controlador ni interfaz que las use, se omiten del diagrama para que este refleje fielmente el sistema que de verdad funciona hoy.



Dentro del dominio de Membresías, la entidad central es membresia, que referencia al socio, al plan contratado y a su estado actual. De ella cuelgan tres tablas de historia: historial_membresia, que registra cada cambio de estado; congelamiento_membresia, que registra cada congelamiento con sus fechas y el número de días que efectivamente sumó al vencimiento; y membresia_sucursal, una tabla puente que permite que una misma membresía dé acceso a más de una sucursal del gimnasio.

En el dominio de Pagos, cada membresía puede tener varias facturas a lo largo del tiempo (una por cada renovación), y cada factura puede saldarse con uno o varios comprobantes de pago —exactamente ese diseño de uno a muchos entre factura y comprobante_pago es lo que permite el pago en cuotas: una factura que no se liquida de una sola vez simplemente acumula más de un comprobante hasta que su saldo llega a cero. Las facturas, a su vez, pueden referenciar opcionalmente una promoción vigente que se aplicó al momento del cobro.

3.3 Procesos de negocio principales

Proceso 1 — Alta de socio, asignación de plan, pago y check-in

Este es el flujo de aceptación principal del sistema, el que se recorre durante la demostración en vivo. Comienza cuando Recepción da de alta a un socio nuevo con sus datos personales, lo que crea automáticamente su cuenta de usuario con rol CLIENTE. A continuación, Recepción le asigna un plan de membresía; el sistema calcula sola la fecha de vencimiento sumando la duración del plan a la fecha de inicio, y la membresía queda en estado ACTIVA. Recepción registra entonces el pago correspondiente; si el pago cubre el total, la factura queda PAGADA de inmediato, y si no, queda PENDIENTE con un saldo que podrá abonarse después. Finalmente, cuando el socio llega físicamente al gimnasio, Recepción registra su entrada, momento en el que el sistema verifica —sin intervención humana— que la membresía esté activa y vigente antes de permitir el acceso; al retirarse, se registra también su salida.

Proceso 2 — Congelamiento de membresía

Este proceso corresponde a uno de los dos ajustes finales pedidos por el Product Owner. Recepción o el Administrador seleccionan una membresía activa y solicitan congelarla hasta una fecha determinada, indicando un motivo. El sistema valida automáticamente tres reglas antes de aceptar la solicitud: que el rango de fechas no se traslape con otro congelamiento ya registrado para esa misma membresía, que el número de días esté dentro del mínimo y el máximo configurados por el gimnasio, y que no se supere el número máximo de congelamientos permitidos por año. Si la solicitud pasa esas validaciones, la membresía cambia a estado CONGELADA y su fecha de vencimiento se recorre exactamente esos días hacia adelante. Si más tarde se decide descongelarla antes de la fecha programada, el sistema descuenta del vencimiento los días que no llegaron a usarse, de modo que el socio nunca paga de más ni de menos por el tiempo que su membresía estuvo congelada.

Proceso 3 — Pago en cuotas (abonos)

Este es el segundo ajuste final del Product Owner. Cuando un pago registrado no alcanza a cubrir el total de una factura, esta queda en estado PENDIENTE con un saldo visible tanto para el personal como para el propio socio a través de su estado de cuenta. Recepción puede entonces registrar abonos adicionales contra esa factura específica; cada abono no puede superar el saldo restante, y el saldo se recalcula después de cada uno. Cuando el saldo llega exactamente a cero, la factura pasa automáticamente a estado PAGADA sin que nadie tenga que marcarla manualmente. Si se llega a la fecha límite sin haber saldado la factura, el sistema la identifica como parte del saldo vencido dentro del estado de cuenta del socio.

4. Historial de Sprints

Esta sección documenta, sprint por sprint, qué se entregó, qué cambios pidió el Product Owner y qué problemas se presentaron durante el desarrollo junto con la forma en que se resolvieron. Para los Sprints 3, 4 y 6, el relato proviene de la experiencia directa del equipo trabajando sobre el proyecto; para los Sprints 1, 2 y 5, se reconstruyó a partir del historial real de mensajes de commit de ambos repositorios, usado como evidencia de lo que efectivamente se construyó y en qué orden.

Sprint 1 — Fundamentos y autenticación (30 de julio – 29 de agosto)

Este primer sprint sentó las bases técnicas sobre las que se construyó todo lo demás. Se definió la estructura inicial tanto del backend como del frontend, y se configuró Spring Security con autenticación basada en JWT, soporte de CORS para que el frontend pudiera consumir la API desde un origen distinto, y documentación automática con Swagger. Se crearon las entidades y migraciones de Rol y Usuario, los DTOs de petición y respuesta, los mappers correspondientes y un manejador global de excepciones que desde entonces centraliza cómo el backend comunica los errores al cliente. El sprint cerró con los endpoints de autenticación, usuarios y roles ya funcionando, y con el servicio de envío de correo (sobre la API de Resend) integrado para las notificaciones que vendrían más adelante.

No se registraron cambios de alcance por parte del Product Owner durante este sprint, ni problemas documentados explícitamente en el historial de commits: fue, en ese sentido, un sprint de arranque sin sobresaltos.

Sprint 2 — Socios, planes y membresías (30 de agosto – 10 de septiembre)

Con la autenticación ya resuelta, este sprint construyó el núcleo funcional del sistema. Se implementó la lógica completa de Socios —registro, listado paginado, filtros de búsqueda y un endpoint dedicado para que el propio socio consulte sus datos—, y un catálogo de Planes de Membresía con operaciones de creación, edición, consulta y documentación en Swagger. Se agregaron las migraciones que introdujeron los planes y las membresías activas iniciales, así como el soporte para que una membresía pudiera asociarse a varias sucursales al mismo tiempo. En paralelo se construyó la entidad Bitácora y el servicio de auditoría que desde entonces registra cada cambio relevante sobre una membresía o una sucursal, y se implementó una primera versión del módulo de Asistencia. El sprint terminó con pruebas unitarias para Socio, Membresía y Plan ya integradas al proyecto.

Tampoco en este sprint se registraron cambios del Product Owner ni problemas documentados de forma explícita; el historial de commits muestra un avance sostenido y sin interrupciones notorias.

Sprint 3 — Automatización (CI/CD)

El foco de este sprint fue dejar de depender de pasos manuales para levantar y desplegar el sistema. Se completó la dockerización de los tres servicios —backend, frontend y base de datos— con un archivo docker-compose capaz de levantar todo desde cero, y se construyeron los dos pipelines de GitHub Actions: uno de integración continua que compila el proyecto y corre su batería de pruebas en cada push o pull request, y uno de despliegue continuo que, al pasar la integración continua en la rama principal, se conecta por SSH a la instancia de AWS EC2 y despliega la versión nueva automáticamente.

Dos problemas marcaron este sprint. El primero fue de documentación: el README del proyecto hacía referencia a un archivo docker-compose.dev.yml que no existía con ese nombre en el repositorio real —

los archivos se llamaban compose.dev.yaml y compose.win.yaml—, lo que generó confusión al tratar de levantar el entorno de desarrollo en Windows hasta que se identificó y corrigió la referencia correcta. El segundo, más costoso en tiempo, fue de infraestructura: los primeros intentos de desplegar a la instancia EC2 fallaban al conectar por SSH sin ningún mensaje de error claro. Usando la AWS CLI para diagnosticar directamente los recursos de red se encontró que el Security Group con las reglas correctas —puertos 22, 80, 8080 y 4200 abiertos— sí existía en la cuenta, pero sencillamente no estaba asociado a la interfaz de red de la instancia; una vez asociado, el despliegue funcionó de inmediato.

Sprint 4 — Incremento de funcionalidades (Pagos y vencimientos)

Este sprint introdujo el dinero al sistema. Se construyeron las entidades Pago, Factura y ComprobantePago de punta a punta: registro del pago, recálculo automático de la fecha de vencimiento de la membresía, historial de pagos consultable con filtros, y generación de un comprobante oficial en PDF usando la librería OpenPDF. En paralelo se implementó la detección de membresías próximas a vencer, con un aviso visible dentro de la propia aplicación, y se integraron al pipeline de integración continua las primeras pruebas automatizadas que verifican específicamente el cálculo de la nueva fecha de vencimiento.

El hallazgo más importante de este sprint, y el que más tiempo absorbió fuera de lo planificado, fue una vulnerabilidad de seguridad de tipo IDOR (Insecure Direct Object Reference): ni el endpoint de descarga del comprobante en PDF ni el de historial de pagos verificaban que la persona que hacía la solicitud fuera realmente la dueña de ese pago, de modo que cualquier socio autenticado podía ver el comprobante de otro socio con solo cambiar el número de identificador en la dirección de la petición. Se corrigió agregando una verificación explícita de propiedad en el controlador —que permite a un Administrador o Recepcionista consultar cualquier pago, pero restringe a un Cliente a consultar únicamente los suyos— y, de paso, quedó como lección para revisar este tipo de control de acceso en cada endpoint nuevo que exponga datos de un socio en particular.

Sprint 5 — Curva del Product Owner y reportes iniciales (9 – 20 de octubre)

Durante este sprint el Product Owner pidió dos cosas que no estaban en el alcance original del módulo de Pagos: la posibilidad de aplicar descuentos a ciertos tipos de socio —estudiantes, adultos mayores, convenios corporativos— y un recargo configurable cuando el pago se realiza fuera de plazo. Ambas reglas se integraron directamente al cálculo del cobro, de modo que el monto final que ve Recepción ya refleja el descuento o el recargo correspondiente antes de confirmar el pago. En paralelo se implementaron el soft delete y la reactivación de usuarios y socios con sus respectivos filtros de consulta, se estabilizaron los recordatorios automáticos de vencimiento por correo, se construyó una primera versión del módulo de Reportes, y se agregaron métricas desagregadas por periodo y por sucursal.

Dos incidentes técnicos se presentaron en este sprint. Varios integrantes del equipo modificaron al mismo tiempo código relacionado con Socio, lo que dejó inestable una de las pruebas de actualización de socio; se resolvió estabilizando los datos de prueba (stubs) que esa prueba utilizaba. Además, durante un merge entre ramas se perdió accidentalmente un método del repositorio de membresías —findMembresiasParaRecordatorio— que los recordatorios automáticos necesitaban para funcionar; se detectó el mismo día y se restauró en un commit posterior.

Sprint 6 — Cierre funcional

El último sprint cerró los dos ajustes finales que el Product Owner había definido en la sesión de refinamiento previa: congelamiento de membresía, con sus reglas de negocio y la extensión automática

del vencimiento, y pago en cuotas mediante abonos, con control de saldo pendiente y estado de cuenta por socio. Ambas funcionalidades ya existían completas del lado del backend desde sprints anteriores, de modo que el trabajo de este sprint se concentró en construir la interfaz de usuario correspondiente: botones de congelar y descongelar integrados tanto en la pantalla de Gestión de Membresías como en la de Gestión de Pagos, una sección de estado de cuenta con la opción de registrar abonos, y un aviso de saldo pendiente visible en el portal del propio socio. También se completaron en este sprint los cinco reportes finales con datos reales, los manuales de usuario y técnico con capturas de pantalla de cada módulo, y los scripts de respaldo y restauración de la base de datos.

El problema técnico más significativo del sprint fue, de nuevo, de infraestructura: la instancia EC2, con apenas 1 GB de RAM, se quedaba sin memoria y dejaba de responder incluso por SSH cuando se intentaba compilar el backend y el frontend al mismo tiempo directamente en el servidor. Se resolvió migrando el flujo de despliegue continuo para que la imagen de Docker se compile en los servidores de GitHub Actions —que no tienen esa limitación— y se publique en GitHub Container Registry; el servidor de producción pasó a limitarse a descargar la imagen ya construida. De forma relacionada, se detectó que el paquete de la imagen del frontend en ese registro había quedado configurado como privado por defecto, a diferencia del backend, que era público, lo que bloqueaba una descarga manual sin autenticación durante las pruebas; quedó documentado como una diferencia de configuración a revisar. Por último, un reinicio de la instancia a mitad de un proceso de compilación interrumpió un despliegue en curso, que hubo que relanzar desde cero.

5. Retrospectiva general del equipo

Qué funcionó

- Automatizar el despliegue, compilando las imágenes de Docker en GitHub Actions en vez de en la instancia de producción, evitó que el servidor —con recursos muy limitados— se convirtiera en un cuello de botella para el equipo durante los últimos sprints.
- Dividir el trabajo por módulo de negocio, con cada funcionalidad viviendo en su propio controlador, servicio y carpeta tanto en el backend como en el frontend, permitió que varios integrantes avanzaran en paralelo durante casi todo el semestre sin pisarse el trabajo.
- Definir los roles del sistema (Administrador, Recepcionista, Entrenador, Socio) y validarlos en el backend desde el primer sprint evitó tener que rediseñar la lógica de permisos más adelante, cuando ya había muchas más pantallas dependiendo de ella.

Qué no funcionó

- Un Security Group mal asociado a la instancia EC2 bloqueó el primer despliegue automático durante varias horas sin ningún mensaje de error que apuntara directamente a la causa.
- La instancia de producción se quedó sin memoria al intentar compilar backend y frontend a la vez directamente en el servidor, deteniendo el progreso del equipo hasta que se migró la compilación a GitHub Actions.
- Dos endpoints con una falla de seguridad de tipo IDOR llegaron a estar activos en el sistema antes de que una revisión posterior los detectara, lo que indica que faltó un paso de revisión de seguridad más sistemático antes de dar por cerrada cada funcionalidad.

Lecciones aprendidas

- Probar la conectividad y verificar los recursos reales del servidor de despliegue desde las primeras semanas del proyecto, en vez de descubrir sus límites bajo presión de tiempo cerca de la entrega final.
- Incorporar una revisión de permisos y de acceso a los datos como parte de la definición de "terminado" de cada funcionalidad nueva, en lugar de tratarla como una auditoría aparte al final del proyecto.
- Documentar explícitamente, en el propio README, cualquier cambio en los nombres de archivos de configuración (como ocurrió con los archivos de Docker Compose), para que el resto del equipo no pierda tiempo buscando un archivo que ya no existe con ese nombre.

Reparto de roles

El equipo mantuvo los mismos roles y responsabilidades a lo largo de todo el proyecto; no hubo rotación de roles entre los integrantes durante el semestre.

Qué harían distinto

- Definir desde el primer sprint de infraestructura los límites reales del servidor de despliegue y diseñar el pipeline de CI/CD para compilar fuera de él, en vez de llegar a esa solución recién en el último sprint tras un problema de memoria.
- Reservar tiempo explícito, dentro de la estimación de cada sprint que incluya despliegue, para la depuración de infraestructura (red, recursos del servidor), ya que fue la causa de desviación que más se repitió a lo largo del proyecto.

6. Estimación de tiempos y comparación con el desarrollo real

La siguiente tabla desglosa, tarea por tarea, el tiempo estimado en tres escenarios optimista, probable y pesimista, medidos en días de trabajo efectivo frente al tiempo que realmente tomó completar cada una, aproximado a partir de las fechas de los commits asociados en ambos repositorios. Es importante señalar que estas estimaciones no se registraron formalmente antes de cada sprint en ningún tablero o documento del equipo: se presentan aquí como una reconstrucción retrospectiva razonable dado el alcance real de cada tarea, no como el plan original tal como se habría documentado en su momento.

Tarea	Requerimiento	Optim.	Prob.	Pesim.	Real	Desviación y causa
Configurar Spring Security + JWT + CORS + Swagger	RF-01	2	3	5	3	Dentro de lo probable
Entidades y migraciones de Usuario/Rol	RF-01, RF-02	1	2	3	2	Dentro de lo probable
Endpoints de autenticación, usuarios y roles	RF-01, RF-02	2	3	4	3	Dentro de lo probable
CRUD de Socios con paginación y filtros	RF-03	2	3	5	4	+1 día; soporte multisucursal se integró en el mismo sprint
Catálogo y CRUD de Planes de Membresía	RF-04	1	2	3	2	Dentro de lo probable
Asignación de membresía con cálculo de vencimiento	RF-05	2	3	4	3	Dentro de lo probable
Auditoría (Bitácora) y multisucursal	RF-20, RF-21	2	3	5	4	Dentro de lo probable
Dockerizar backend, frontend y base de datos	—	1	2	3	2	Dentro de lo probable
Configurar CI (build + pruebas) en GitHub Actions	—	1	2	3	2	Dentro de lo probable
Configurar CD (despliegue automático a EC2)	—	1	2	4	5	+1 día; el Security Group de la instancia no estaba asociado a la red correcta y tomó varias horas diagnosticarlo
Registro de pagos, factura y comprobante PDF	RF-08, RF-11	2	3	5	4	Dentro de lo probable
Panel de membresías próximas a vencer	RF-17, RF-18	1	2	4	3	Dentro de lo probable

Corregir vulnerabilidad IDOR en endpoints de pago	—	0	1	2	3	+2 días; no estaba planificada, se encontró en una revisión posterior al sprint
Soft delete y reactivación de usuarios/socios	RF-03	1	2	3	2	Dentro de lo probable
Promociones y recargo por pago tardío	RF-22, RF-23	2	3	5	5	Dentro de lo probable
Primera versión de Reportes y métricas	RF-24	2	4	6	5	Dentro de lo probable
Estabilizar pruebas tras cambios simultáneos de varios integrantes	—	0	1	2	2	Dentro de lo probable, aunque no estaba contemplada como tarea aparte
UI de Congelamiento integrada a Pagos y Membresías	RF-07	2	3	5	4	Dentro de lo probable
UI de Abonos y estado de cuenta	RF-09, RF-10	2	3	5	4	Dentro de lo probable
Manuales de usuario y técnico con capturas	—	1	2	3	2	Dentro de lo probable
Scripts de respaldo y restauración de la base de datos	—	0	1	2	1	Dentro de lo probable

Las causas de desviación más frecuentes a lo largo del proyecto fueron, en orden de impacto: problemas de infraestructura del servidor de despliegue —tanto de red como de memoria—, y tiempo no planificado para corregir fallas de seguridad encontradas sobre la marcha en lugar de durante el desarrollo original de la funcionalidad.

Acciones para estimar mejor en proyectos futuros

- Reservar explícitamente un margen de tiempo para depuración de infraestructura (conectividad de red, memoria del servidor) en cualquier sprint que incluya trabajo de despliegue, dado que fue la causa de desviación más repetida.
- Incluir la revisión de permisos y de acceso a los datos de cada endpoint nuevo dentro de la estimación de la tarea que lo crea, en vez de absorberla como una desviación no planificada de un sprint posterior.

7. Conclusiones

Al cierre del semestre, Claude Lovers Gym cubre el ciclo completo que se propuso al inicio del proyecto: un socio puede darse de alta, se le puede asignar un plan, se le puede cobrar —de contado o en cuotas— , puede congelar su membresía si lo necesita, y puede marcar su entrada y salida del gimnasio, todo bajo un sistema de roles validado en el servidor y desplegado de forma automática cada vez que el código cambia en la rama principal. El camino para llegar hasta ahí no fue lineal: hubo decisiones de arquitectura que se revisaron sobre la marcha, como mover la compilación de las imágenes de Docker fuera del servidor de producción, y hubo errores reales, como la vulnerabilidad de seguridad detectada en el módulo de pagos, que obligaron a dedicar tiempo no planificado a corregirlos.

Más que un sistema terminado de forma perfecta, lo que este documento intenta mostrar es un proceso honesto: qué se construyó en cada sprint, qué salió mal, cómo se resolvió, y qué haría el equipo distinto si volviera a empezar. Esa honestidad —documentar también los incidentes de un Security Group mal configurado o de una instancia que se quedó sin memoria a mitad de un despliegue— es, para el equipo, tan parte de la entrega final como el propio código funcionando en producción.